

Lösungsblatt: Protokolle und Schnittstellen

Modul 321 – Lektion 08

Hinweis: Dieses Lösungsblatt enthält Musterlösungen und einen möglichen Erwartungshorizont. Andere Antworten sind korrekt, wenn sie fachlich sauber begründet sind und die Rollen der Protokolle und Schnittstellen nachvollziehbar eingeordnet werden.

1. Begriffe erklären

Begriff	Mögliche Lösung
TCP	TCP ist ein verbindungsorientiertes Transportprotokoll, das eine zuverlässige und geordnete Datenübertragung garantiert. Verlorene Pakete werden automatisch wiederholt.
HTTP	HTTP ist ein Protokoll für den Austausch von Nachrichten zwischen Client und Server. Es baut auf TCP auf und verwendet Anfragen mit Methoden wie GET oder POST sowie Antwort-Status-Codes.
REST	REST ist ein Architekturstil für APIs, der HTTP-Methoden und Ressourcen-URLs nutzt. Jede Anfrage ist unabhängig (stateless), und Daten werden meist als JSON übertragen.
GraphQL	GraphQL ist eine Abfragesprache für APIs, bei der der Client genau angibt, welche Daten er benötigt. Alle Anfragen laufen über einen einzigen Endpunkt.
gRPC	gRPC ist ein Framework für schnelle, typsichere Kommunikation zwischen Diensten. Es verwendet das binäre Protobuf-Format und setzt auf HTTP/2.
WebSocket	WebSockets ermöglichen eine dauerhafte, bidirektionale Verbindung zwischen Client und Server. Beide Seiten können jederzeit Nachrichten senden, ohne erneuten Verbindungsaufbau.
Gateway	Ein API-Gateway ist ein einzelner Eintrittspunkt für alle Anfragen. Es übernimmt Aufgaben wie Authentifizierung, Routing und Rate Limiting zentral.
Proxy	Ein Proxy ist ein Vermittler im Netzwerk. Ein Forward Proxy steht vor dem Client und leitet dessen Anfragen weiter; ein Reverse Proxy steht vor dem Server und nimmt Anfragen entgegen.

2. Beispiele zuordnen

1. REST
2. gRPC
3. WebSocket
4. GraphQL
5. Gateway
6. TCP
7. Proxy (Forward Proxy)
8. HTTP

3. Protokolle und Schnittstellen vergleichen

Frage	REST	GraphQL
Wie viele Endpunkte typischerweise?	Viele, je eine URL pro Ressource (z. B. /users, /orders)	Ein einziger Endpunkt (/graphql)
Wer bestimmt die Datenstruktur?	Der Server – er liefert eine feste Antwortstruktur	Der Client – er formuliert die Query selbst
Geeignetes Szenario?	Öffentliche API, einfache CRUD-Operationen	Mobile Apps, wechselnde Datenanforderungen

Frage	Gateway	Reverse Proxy
Hauptaufgabe?	Einheitlicher Zugangspunkt, Routing zu internen Diensten, Sicherheitslogik zentral	Anfragen vor dem Server entgegennehmen, weiterleiten, SSL und Caching übernehmen
Was vereinfacht es?	Clients müssen nur eine Adresse kennen; Sicherheit und Routing sind zentral geregelt	Server werden nach aussen verborgen; SSL und Lastverteilung müssen nicht pro Dienst eingerichtet werden
Warum nicht verwechseln?	Ein Gateway ist fachlich-orientiert und kennt Dienste; ein Reverse Proxy ist technisch und leitet transparent weiter	Gleiches Argument: Aufgaben überschneiden sich, aber die Verantwortung ist eine andere

4. Aussagen beurteilen

1. Falsch. HTTP ist ein Transportprotokoll; REST ist ein Architekturstil, der HTTP nutzt. REST setzt HTTP voraus, ist aber nicht identisch damit.

2. Falsch. GraphQL verwendet genau einen Endpunkt. Der Client steuert über Queries, welche Daten zurückgeliefert werden.
3. Falsch. gRPC ist in vielen Szenarien schneller, aber nicht grundsätzlich. Bei einfachen Anfragen oder öffentlichen APIs kann REST besser passen.
4. Richtig. Ein API-Gateway kann Authentifizierungslogik zentral übernehmen, damit einzelne Dienste diese nicht selbst implementieren müssen.
5. Falsch. WebSockets sind für dauerhafte, bidirektionale Kommunikation ausgelegt. Für einmalige Leseanfragen ist REST oder HTTP einfacher und ausreichend.
6. Richtig. TCP garantiert, dass Pakete geordnet und vollständig ankommen; verlorene Pakete werden wiederholt gesendet.

5. Szenarien analysieren

1. REST. Öffentliche APIs mit klaren Ressourcen und CRUD-Operationen sind der klassische Anwendungsfall für REST.
2. GraphQL. Eine mobile App, die je nach Ansicht unterschiedliche Datenmengen braucht, profitiert davon, dass der Client die Datenstruktur selbst bestimmt.
3. gRPC. Interne Microservice-Kommunikation profitiert von der Performance und Typsicherheit, die gRPC mit Protobuf bietet.
4. WebSocket. Eine kollaborative Anwendung braucht dauerhaften, bidirektionalen Datenaustausch in Echtzeit.
5. Gateway. Ein API-Gateway übernimmt den einheitlichen Zugangspunkt und steuert das Routing zu den verschiedenen Diensten.

6. Schnittstellen einordnen

Stil	Datenformat	Kommunikationsmuster	Typischer Einsatz
REST	JSON	Request / Response	Web-APIs, öffentliche Schnittstellen
GraphQL	JSON	Request / Response (flexible Query)	Mobile Apps, Aggregation von Daten
gRPC	Protobuf (binär)	Streaming möglich, Request / Response	Interne Microservices
WebSocket	Beliebig (Text, Binär)	Bidirektional, dauerhafte Verbindung	Chat, Live-Daten, Echtzeit

7. Mini-Entwurf

1. Mögliche Wahl: REST für Benutzeranmeldung und Bestellverwaltung, GraphQL für die mobile App, gRPC für interne Dienste (Lager, Bezahlung, Versand), WebSocket für Live-Updates der Fahrer, Gateway als einheitlicher Zugangspunkt.

2. REST ist für Benutzeranmeldung und Bestellungen ausreichend einfach. GraphQL vermeidet unnötigen Datentransfer auf Mobilgeräten. gRPC bietet die nötige Performance zwischen internen Diensten. WebSockets sind nötig, weil Fahrer sofort reagieren müssen. Das Gateway verbirgt die interne Komplexität vor externen Clients.
3. Der Zugang von aussen wird durch das API-Gateway gesteuert.

8. Reflexion

1. Klar definierte Schnittstellen erlauben es verschiedenen Teams, unabhängig voneinander zu arbeiten. Änderungen können beherrschbar durchgeführt werden, ohne alle Clients gleichzeitig anpassen zu müssen.
2. Komplexere Technologien bringen auch mehr Betriebsaufwand, Lernkurve und Fehlerquellen mit sich. Oft reicht ein einfacherer Ansatz vollständig aus.
3. Ein Protokoll legt technische Regeln für den Datentransport fest. Ein Architekturstil wie REST beschreibt, wie man eine Schnittstelle strukturiert und nutzt, und setzt typischerweise ein Protokoll wie HTTP voraus.